

Research Notes on Perceptron

Perceptron

$$y = \text{sign}(w \cdot x + b)$$

$$w \leftarrow w + \eta(y - \hat{y})x$$

>> Section 1

Variants The pocket algorithm with ratchet (Gallant, 1990) solves the stability problem of perceptron learning by keeping the best solution seen so far "in its pocket". The pocket algorithm then returns the solution in the pocket, rather than the last solution. It can be used also for non-separable data sets, where the aim is to find a perceptron with a small number of misclassifications. However, these solutions appear purely stochastically and hence the pocket algorithm neither approaches them gradually in the course of learning, nor are they guaranteed to show up within a given number of learning steps. The Maxover algorithm (Wendemuth, 1995) is "robust" in the sense that it will converge regardless of (prior) knowledge of linear separability of the data set. In the linearly separable case, it will solve the training problem – if desired, even with optimal stability (maximum margin between the classes). For non-separable data sets, it will return a solution with a computable small number of misclassifications. In all cases, the algorithm gradually approaches the solution in the course of learning, without memorizing previous states and without stochastic jumps. Convergence is to global optimality for separable data sets and to local optimality for non-separable data sets. The Voted Perceptron (Freund and Schapire, 1999), is a variant using multiple weighted perceptrons. The algorithm starts a new perceptron every time an example is wrongly classified, initializing the weights vector with the final weights of the last perceptron. Each perceptron will also be given another weight corresponding to how many examples do they correctly classify before wrongly classifying one, and at the end the output will be a weighted vote on all perceptrons. In separable problems, perceptron training can also aim at finding the largest separating margin between the classes. The so-called perceptron of optimal stability can be determined by means of iterative training and optimization schemes, such as the Min-Over algorithm (Krauth and Mezard, 1987) or the AdaTron (Anlauf and Biehl, 1989)). AdaTron uses the fact that the corresponding quadratic optimization problem is convex. The perceptron of optimal stability, together with the kernel trick, are the conceptual foundations of the support-vector machine. The α $\{\displaystyle \alpha\}$

-perceptron further used a pre-processing layer of fixed random weights, with thresholded output units. This enabled the perceptron to classify analogue patterns, by projecting them into a binary space. In fact, for a projection space of sufficiently high dimension, patterns can become linearly separable. Another way to solve nonlinear problems without using multiple layers is to use higher-order networks (sigma-pi unit). In this type of network, each element in the input vector is extended with each pairwise combination of multiplied inputs (second order). This can be extended to an n-order network. A generalization of the perceptron model is the Receptron that incorporates non-linear interactions between inputs. A single receptron is able to classify non-linear Boolean functions. It should be kept in mind, however, that the best classifier is not necessarily that which classifies all the training data perfectly. Indeed, if we had the prior constraint that the data come from equi-variant Gaussian distributions, the linear separation in the input space is optimal, and the nonlinear solution is overfitted. Other linear classification algorithms include Winnow, support-vector machine, and logistic regression.

>> Section 2

The following simple proof is due to Novikoff (1962). The idea of the proof is that the weight vector is always adjusted by a bounded amount in a direction with which it has a negative dot product, and thus can be bounded above by $O(\sqrt{t})$, where t is the number of changes to the weight vector. However, it can also be bounded below by $O(t)$ because if there exists an (unknown) satisfactory weight vector, then every change makes progress in this (unknown) direction by a positive amount that depends only on the input vector.

>> Section 3

In machine learning, the perceptron is an algorithm for supervised learning of binary classifiers. A binary classifier is a function that can decide whether or not an input, represented by a vector of numbers, belongs to some specific class. It is a type of linear classifier, i.e. a classification algorithm that makes its predictions based on a linear predictor function combining a set of weights with the feature vector.

>> Section 4

While the perceptron algorithm is guaranteed to converge on some solution in the case of a linearly separable training set, it may still pick any solution and problems may admit many solutions of varying quality. The perceptron of optimal stability, nowadays better known as the linear support-vector machine, was designed to solve this problem (Krauth and Mezard, 1987).

Research Notes on Perceptron

Perceptron

>> Section 5

A single perceptron can learn to classify any half-space. It cannot solve any linearly nonseparable vectors, such as the Boolean exclusive-or problem (the famous "XOR problem"). A perceptron network with one hidden layer can learn to classify any compact subset arbitrarily closely. Similarly, it can also approximate any compactly-supported continuous function arbitrarily closely. This is essentially a special case of the theorems by George Cybenko and Kurt Hornik.

>> Section 6

The binary value of $f(\mathbf{x})$ (0 or 1) is used to perform binary classification on $\mathbf{x}$ as either a positive or a negative instance. Spatially, the bias shifts the position (though not the orientation) of the planar decision boundary. In the context of neural networks, a perceptron is an artificial neuron using the Heaviside step function as the activation function. The perceptron algorithm is also termed the single-layer perceptron, to distinguish it from a multilayer perceptron, which is a misnomer for a more complicated neural network. As a linear classifier, the single-layer perceptron is the simplest feedforward neural network.

>> Section 7

Although the perceptron initially seemed promising, it was quickly proved that perceptrons could not be trained to recognise many classes of patterns. This caused the field of neural network research to stagnate for many years, before it was recognised that a feedforward neural network with two or more layers (also called a multilayer perceptron) had greater processing power than perceptrons with one layer (also called a single-layer perceptron). Single-layer perceptrons are only capable of learning linearly separable patterns. For a classification task with some step activation function, a single node will have a single line dividing the data points forming the patterns. More nodes can create more dividing lines, but those lines must somehow be combined to form more complex classifications. A second layer of perceptrons, or even linear nodes, are sufficient to solve many otherwise non-separable problems. In 1969, a famous book entitled *Perceptrons* by Marvin Minsky and Seymour Papert showed that it was impossible for these classes of network to learn an XOR function. It is often incorrectly believed that they also conjectured that a similar result would hold for a multi-layer perceptron network. However, this is not true, as both Minsky and Papert already knew that multi-layer perceptrons were capable of producing an XOR function. (See the page on *Perceptrons* (book) for more information.) Nevertheless, the often-miscited Minsky and Papert text caused a significant decline in interest and

funding of neural network research. It took ten more years until neural network research experienced a resurgence in the 1980s. This text was reprinted in 1987 as "*Perceptrons - Expanded Edition*" where some errors in the original text are shown and corrected.